Felicity Moyo u24702791
Nico Sibiya u24667642
Chavonne Makurira u2502880

Task 1

EventFlow will be managing and running the operations of Mystifare. At Mystifare, attendees are transported back in time to an authentic old English village, where a rich tapestry of medieval entertainment and activities awaits. Throughout the day, visitors can enjoy musical performances by minstrels and troubadours, theatrical productions ranging from Shakespearean plays to comedic interludes, and thrilling jousting tournaments featuring armored knights in full competition. Artisans and merchants offer period-accurate costumes, handcrafted jewelry, leather goods, and medieval crafts, while a diverse selection of food vendors serve hearty medieval-inspired cuisine, including roasted meats, meat pies, pasta, and wood-fired pizza, alongside specialty drinks like mead and ale.Because The fair runs over the course of two days Mystifare offers tented accommodation for those wishing to extend their visit, blending modern comfort with a period atmosphere, all while dedicated security teams known as The Watch are scattered across the grounds to ensure the safety and well-being of every attendee.

Concrete Event units

- Performance: A single scheduled musical or theatre performance which is affected by schedule changes and evacuation alerts
- FoodVendor: Sells food and drink items, Allergen listings must be made available.
- MerchVendor: Sells costumes, souvenirs and other merchandise. No allergy requirements needed
- Tent: A camping unit able to hold a maximum of two attendees at a time
- The Watch: security detail which manages attendees

Grouping

Mystifare simulates a village and is thus may be divided into four major zones, The North, South, East and West. These zones may contain sub-zones where related event units are found, e.g. Entertainment zone, Accommodation zone etc.

- Major Zone: the top-level composite, A Major zone divides the ‘village’ into quadrants. A Major Zone such as North may contain sub-zones and concrete event units
- Stage: Stage is sub-zone(composite) nested inside a Major Zone. It contains performances and theWatch.

- Accommodation: Accommodation is a sub-zone(composite) nested inside a zone, grouping tent units.

1.2

Composite

Component: EventComponent

Leaf:

- MerchVendor
- FoodVendor
- Performance
- TheWatch
- Tent

Composite: EventZone: inherits from the Component interface and can contain other EventZones or Concrete Event Units

Client: main

Observer

Subject: Control

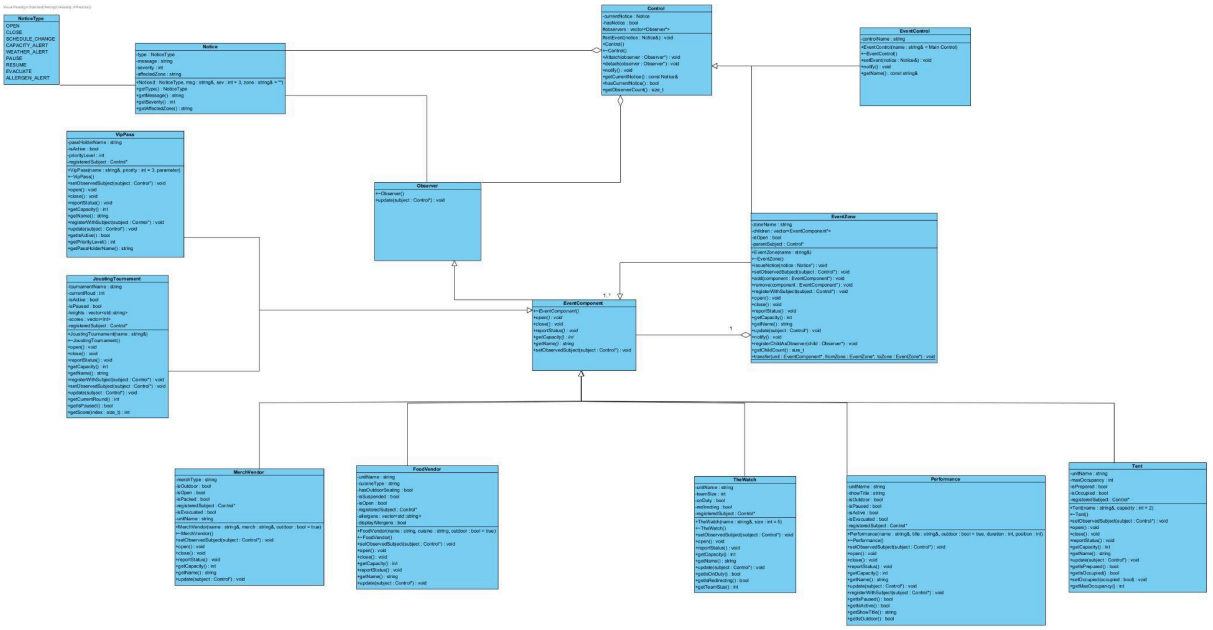
ConcreteSubject: EventControl, EventZone: EventZone inherits from Control it forwards its children notices allowing for cascading notices.

Observer: Observer

ConcreteObserver:

- MerchVendor
- FoodVendor
- Performance
- JoustingTournement
- TheWatch
- Tent
- VipPass
- EventZone: it pulls notices and reacts to them.

1.3



1.4

a.) According to IONOS.com a genuine part-whole tree is a structure where a complex entity is recursively composed of sub-components and both share a common interface. In our design the Composite is a genuine part-whole tree so that the client may treat a Zone and Concrete Event Units the same way without the need for type-checking as they share the same base class EventComponent. It also allows us to create a detailed map of the fair and enables cascading notifications.

b.) Observer is required instead of hard-coded calls to avoid tight coupling and violating the open-close principle. If hard-coded calls were to be used we would have to change existing code every time a dependent class is added, removed or edited. It would also mean “Subject” would call every dependent class which makes the code more difficult to maintain..

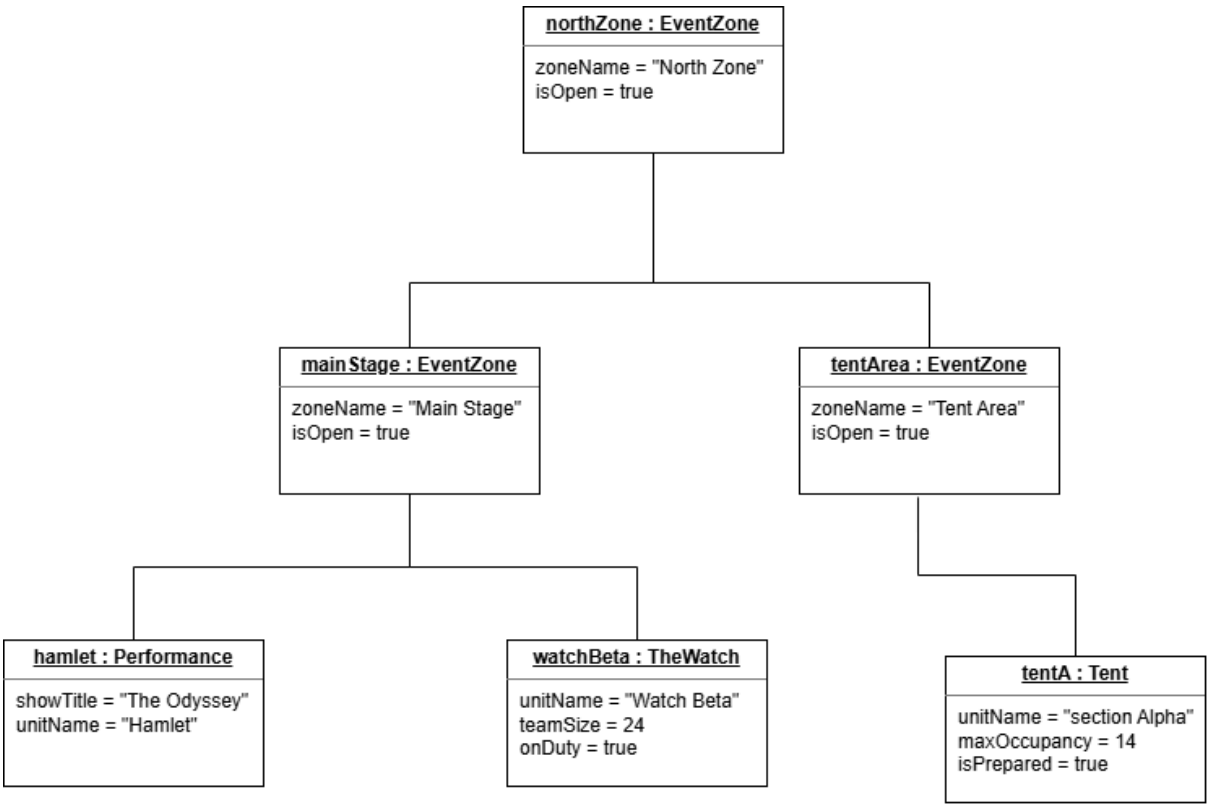
c.) event components are owned by the Composite own meaning EventZone may own other EventZone objects and leaves. It is responsible for deleting its children.

d.) In our design observers references are not owned by any class, they are only pointed to using raw pointers. This is because an Observer can exist independent of the Subject. However if a composite has a leaf which is also an observer then the composite owns the observer.

e.) The roles of Subject and Observer are different,have different hierarchies and happen at different times. A class being both a Subject and Observer is not pattern misuse because it is the same class fulfilling different responsibilities. In our design EventZone needs this to enable cascading notifications.

Task 2

2.3



2.4 EventZone owns its children through its children vector. When the root EventZone is destroyed, its destructor iterates through its children and deletes each child. If a child is itself an EventZone(composite) . Its destructor recursively deletes its own children. Therefore, deleting the root releases the entire owned subtree exactly once . the client must not separately delete owned descendants . For runtime transfer, an object must have exactly one owner at a time. The transfer operation first detaches the object from the old observer subject, removes it from the old composite, adds it to the new composite , updates its observed subject, and reattaches it to the new subject. This ensures that both ownership and observer registration move together without double deletion

Task 3

3.1 The policy is that when an already existing observer is being attached the function returns quickly,ignoring it. When a non-existing observer is being detached it returns and does not erase it from the list so it is ignored.

3.2. The deletion policy is that the observer will detach itself when it is being destroyed because it is simple and safe. The subject cleaning it up would be dangerous if the subject is destroyed before the observer which would destroy the observer which should have an independent lifetime from the subject. Lifetime ordering would also be dangerous since it guarantees that the subject outlives the observer because it is not flexible and fragile for a composite tree where parents delete children.

3.5. We chose the PULL method. The PUSH method is simple to implement and the observer gets all the data immediately in one call. However it has tight coupling because the subject needs to know exactly what observer needs and that is wasteful

as observers do not need certain fields that they receive. The PULL method is decoupled because the subject does not need to know what data each observer wants and it is flexible because observers can choose which fields to pull which is efficient because observers only request the data that they need. However it requires more calls to retrieve the data which adds more code and each observer must specifically query the subject. The state being transferred is of the Notice class that is an object with a NoticeType enumeration, a message,severity and zone.

Task 4

4.1.In the event that there is a weather alert a Performance will Pause and the stage closes whereas indoor performances continue as usual. TheWatch would remain active and redirect attendees to a shelter, an outdoor foodVendor will suspend its service and close down, Tent will prepare for emergency occupancy by opening and checking capacity. The outdoor MerchantVendor will pack up and secure inventory.

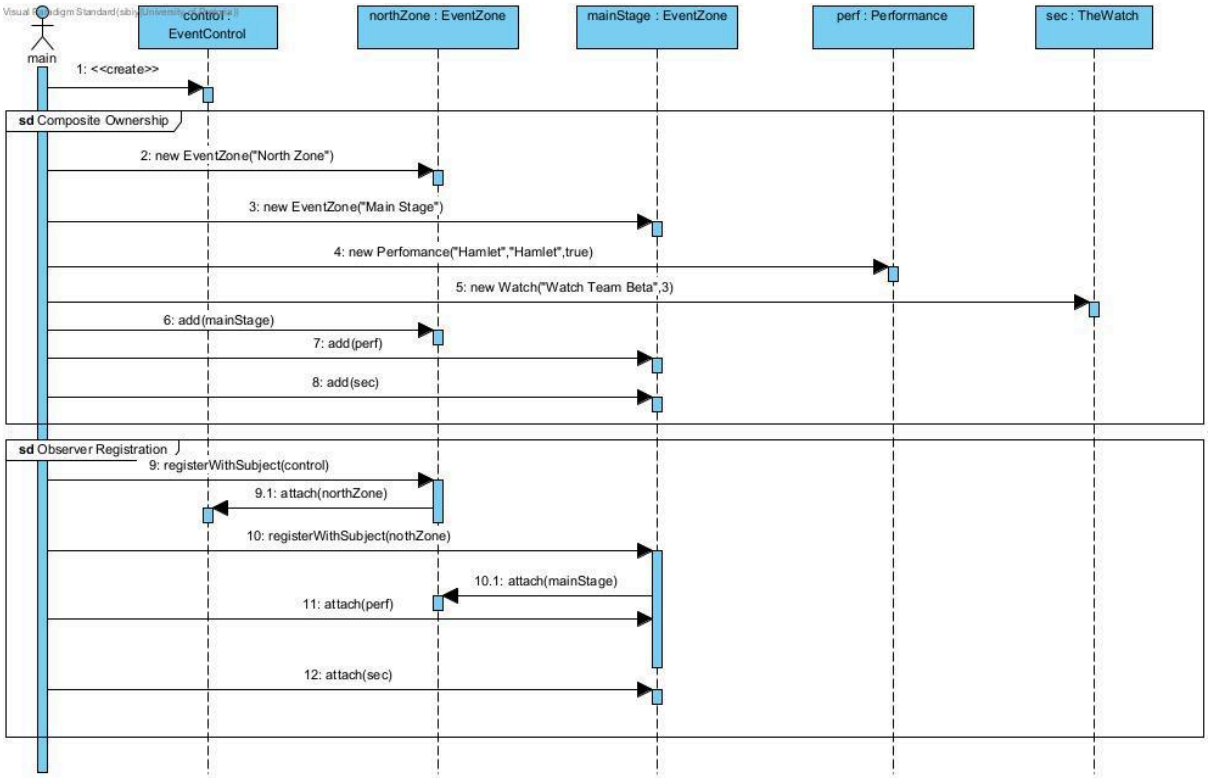
In the event of a resume notice MerchVendors will unpack and open up again, a FoodVendor will no longer be suspended and will open up, a performance will now longer be evacuated and be active again, a tent will no longer prepare for an emergency and the watch will stop redirecting people.

This is possible because all the concrete units inherit from the same abstract class being EventComponent and implement the same Observer Interface with a shared update(Control*) method. When a weather alert is issued the update function is called on every registered observer and it does not check the type of observer or a giant switch statement but polymorphism ensures each unit's own update method is called. Then each unit's internal state determines its specific reaction.

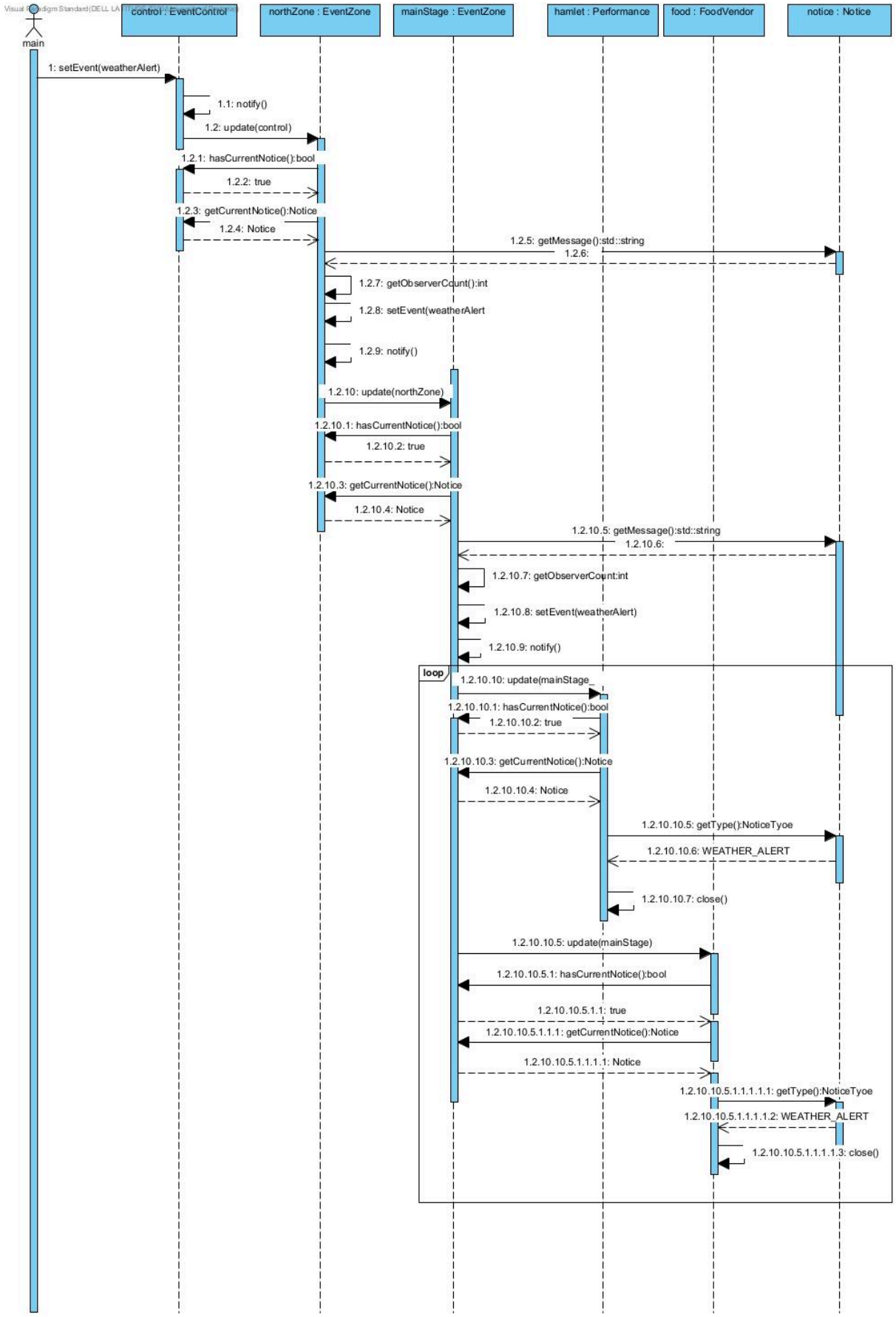
4.3. An example of this when there is weather alert and a performance determines whether it is outdoor or indoor to determine whether it should stop or continue in Performance.h
4.4 . For a Food Vendor we added an allergen alert for certain cuisines and they will be displayed when someone picks that dish
We added a VIP pass system so that when a capacity alert is sent VIPs are given priority entry
We added a Jousting Tournament Scoreboard as a ConcreteObserverin the Observer pattern.

Task 5

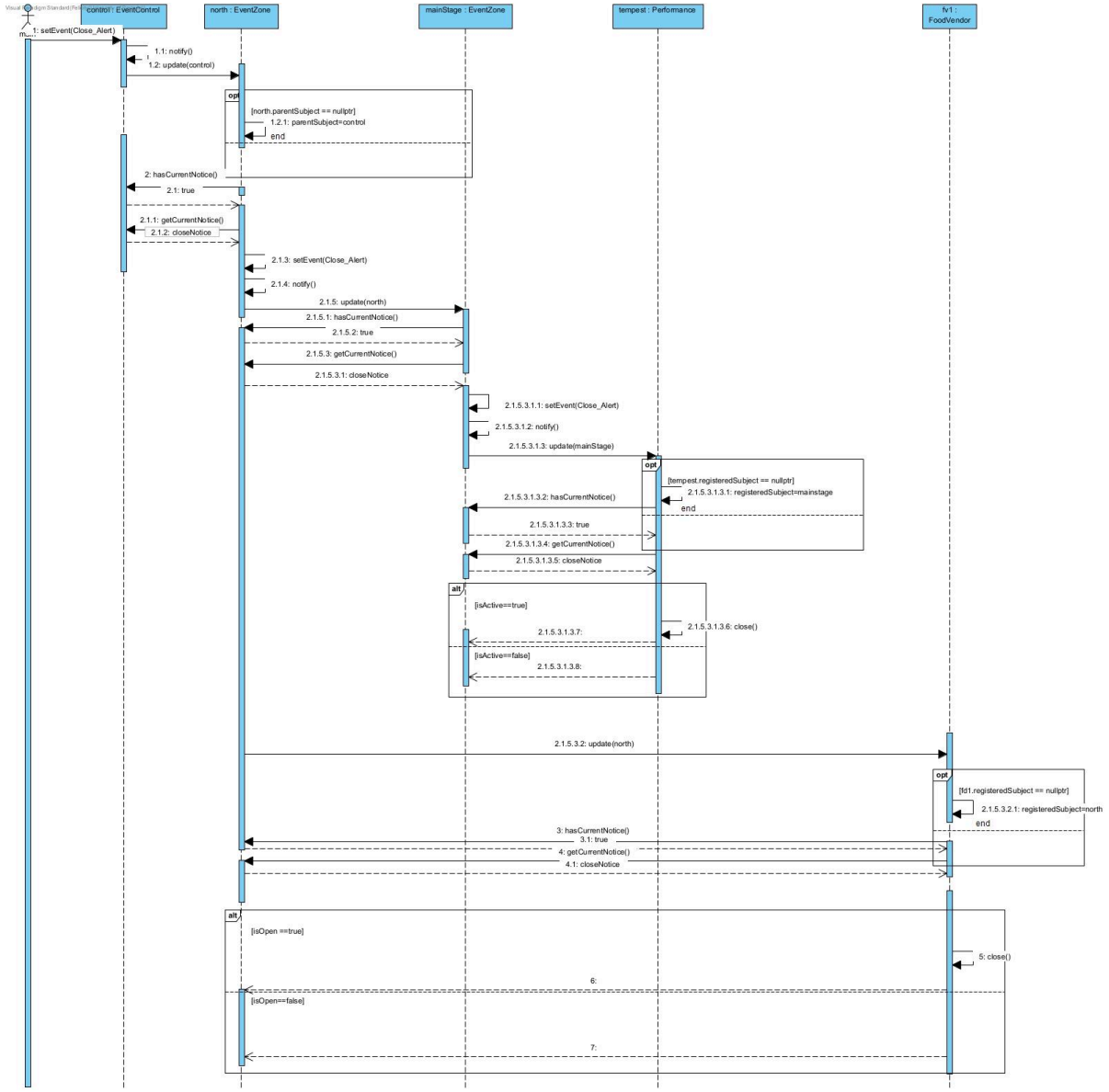
5.1SD1 – Building and registering part of the event.



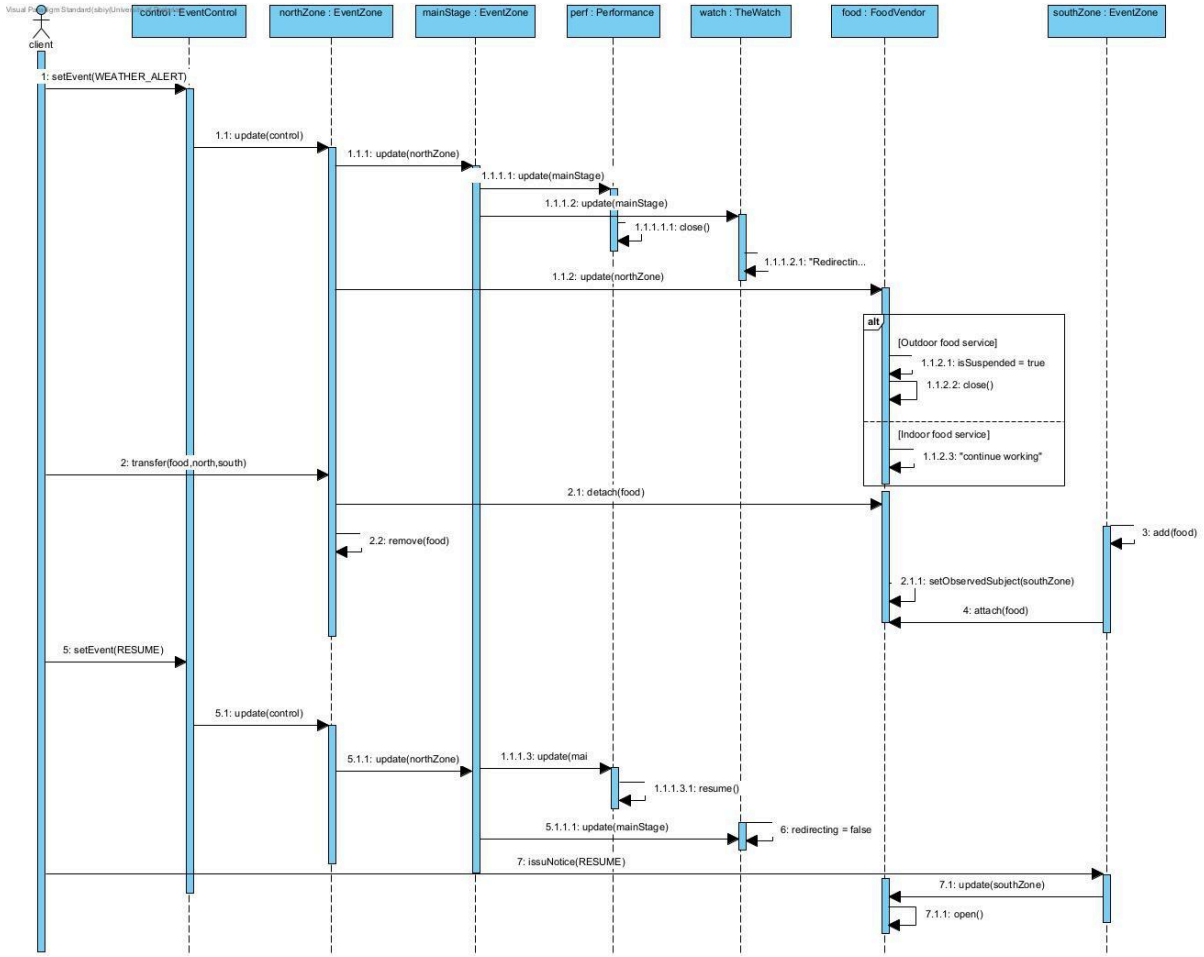
5.2 SD2 – Cascading event notification.



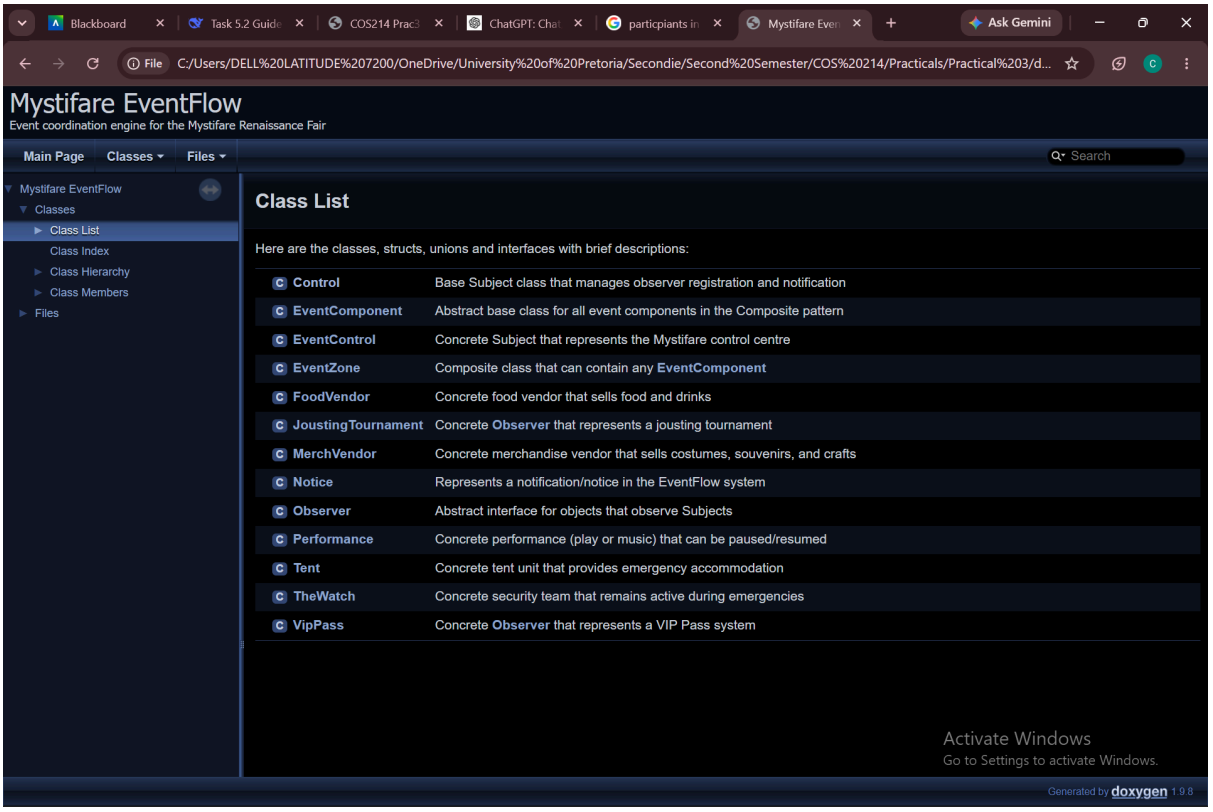
5.3SD3 – Conditional event response and Composite behaviour.



5.4SD4 – Your signature event scenario.


client
control : EventControl
northZone : EventZone
mainStage : EventZone
perf : Performance
watch : TheWatch
food : FoodVendor
southZone : EventZone
1: setEvent(WEATHER_ALERT)
1.1: update(control)
1.1.1: update(northZone)
1.1.1.1: update(mainStage)
1.1.1.2: update(mainStage)
1.1.1.1.1: close()
1.1.1.2.1: "Redirectin...
1.1.2: update(northZone)
alt
[Outdoor food service]
1.1.2.1: isSuspended = true
1.1.2.2: close()
[Indoor food service]
1.1.2.3: "continue working"
2: transfer(food,north,south)
2.1: detach(food)
2.2: remove(food)
3: add(food)
2.1.1: setObservedSubject(southZone)
4: attach(food)
5: setEvent(RESUME)
5.1: update(control)
5.1.1: update(northZone)
1.1.1.3: update(mai
1.1.1.3.1: resume()
5.1.1.1: update(mainStage)
6: redirecting = false
7: issuNotice(RESUME)
7.1: update(southZone)
7.1.1: open()

Doxygen proof



Task 7

7.4 Work was divided according to the allocated marks such that each member completed tasks worth an equal share of the total. Each member completed tasks of up to approximately 85 marks in total. Felicity Moyo was in charge of completing Task 1- planning and architecture (45 marks), Task 5.3- 1 Sequence Diagram (25 marks), Task 8- testing/ main (10 marks) and Task 7.4 Github Workflow Reflection (5 marks). Nico Sibiya was in charge of Task 2- Composite(35 marks), Task 5.1, 5.2 - 2 Sequence Diagrams(50 marks). Chavonne Makurira was in charge of Task 3- Observer(30 marks), Task 5.2- 1 Sequence Diagram (25 marks), Task 6-Doxygen(20 marks). All members remained mindful of Tasks 4 and 7. We coordinated our workflow through a Github repository. We used branches for different sections of work, to avoid committing to the main. Changes were discussed and agreed upon by all members thereafter corrections were done in the branches and once completed were merged with the main. Branches helped us share code without breaking/disrupting the main branch, we were able to work on each other's code independently.The use of branches let us identify which parts of our system were fully functional and which parts were still under development thus making progress tracking much easier. Pull requests were used to merge branches to the main, this helped us resolve any conflicts before making changes thus protecting the main branch.

1. The creation of the testing branch: created a branch which contained the .h files allowed for one member to test the code written by another member, make corrections and discuss changes without affecting their original code.

2. Merge pull request #3 from joyincity/Observer-Notification-System this pull request was made to merge the observer-notification-system branch with the main.

Team Contribution Statement

Team Members and Contributions

Nico Sibiya
Implemented the Composite pattern, including the EventComponent interface and EventZone composite. Worked on the ownership hierarchy and runtime transfer of event units between zones. Assisted with integration, debugging, and verification of the event scenarios. Completed the Building and Registering Part of the Event sequence diagram and the Signature Event Scenario sequence diagram. Completed the team contribution statement and the composite event structure.

Felicity Moyo
Completed the event concept and architecture. Identified five unique event units and three different areas that would appear in the composite tree. Identified the GoF participants of the Composite and Observer patterns in our design. Completed the UML class diagram for the full EventFlow design, showing all abstract classes, inheritance, associations, ownership, Observer registrations, relevant attributes and operations, and a virtual destructor on every polymorphic base. Completed the driver file,main.cpp, the GitHub essay, and the Conditional Event Response and Composite Behaviour sequence diagram. Assisted with integration, debugging, and verification of the event scenarios.

Chavonne Makurira
Implemented several concrete event units, including Performance, FoodVendor, and MerchVendor. Added behaviour for weather alerts, schedule changes, opening and closing, and other event notifications. Implemented the Observer pattern, including Observer, Control, and EventControl, and worked on notification propagation from the control centre through the event hierarchy. Implemented and tested additional event units such as Tent, TheWatch, and JoustingTournement. Assisted with integration, debugging, and verification of the event scenarios. Completed the Observer notification system, the Cascading Event Notification sequence diagram, and the Doxygen documentation.

Team Coordination

The team divided the implementation according to the major requirements of the project while maintaining common interfaces so that the individual components could be integrated effectively. Members regularly discussed the design and reviewed one another's work to ensure that the Composite and Observer patterns were implemented consistently. UML diagrams were developed alongside the implementation and checked against the actual C++ classes and interactions. After integrating the individual components, the team tested notification cascading, ownership relationships, runtime transfers, and event responses together. Problems discovered during testing were discussed as a team, and solutions were implemented collaboratively before the final submission.

https://github.com/joyincity/COS-214-practical-3